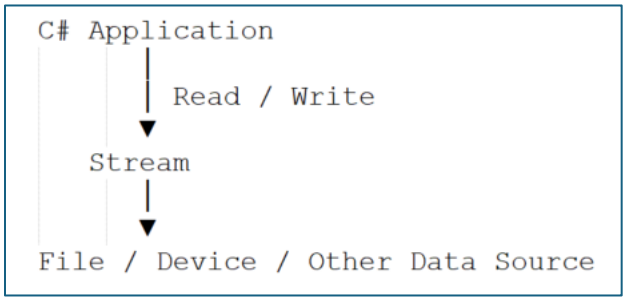
C# File Input and Output

File input/output (I/O) allows an application to **persist data outside the program**, retrieve it later, and interact with files and directories on the operating system.

In C#, file I/O is primarily supported through the **`System.IO`** namespace. This section focuses on:

- streams;
- I/O exception handling;
- resource management and **`IDisposable`**;
- **`File`**;
- **`StreamReader`** and **`StreamWriter`**;
- **`FileStream`**;
- **`Directory`** and **`DirectoryInfo`**.

A useful overall model is:



A **stream** provides the abstraction through which data moves between the application and another location.

---

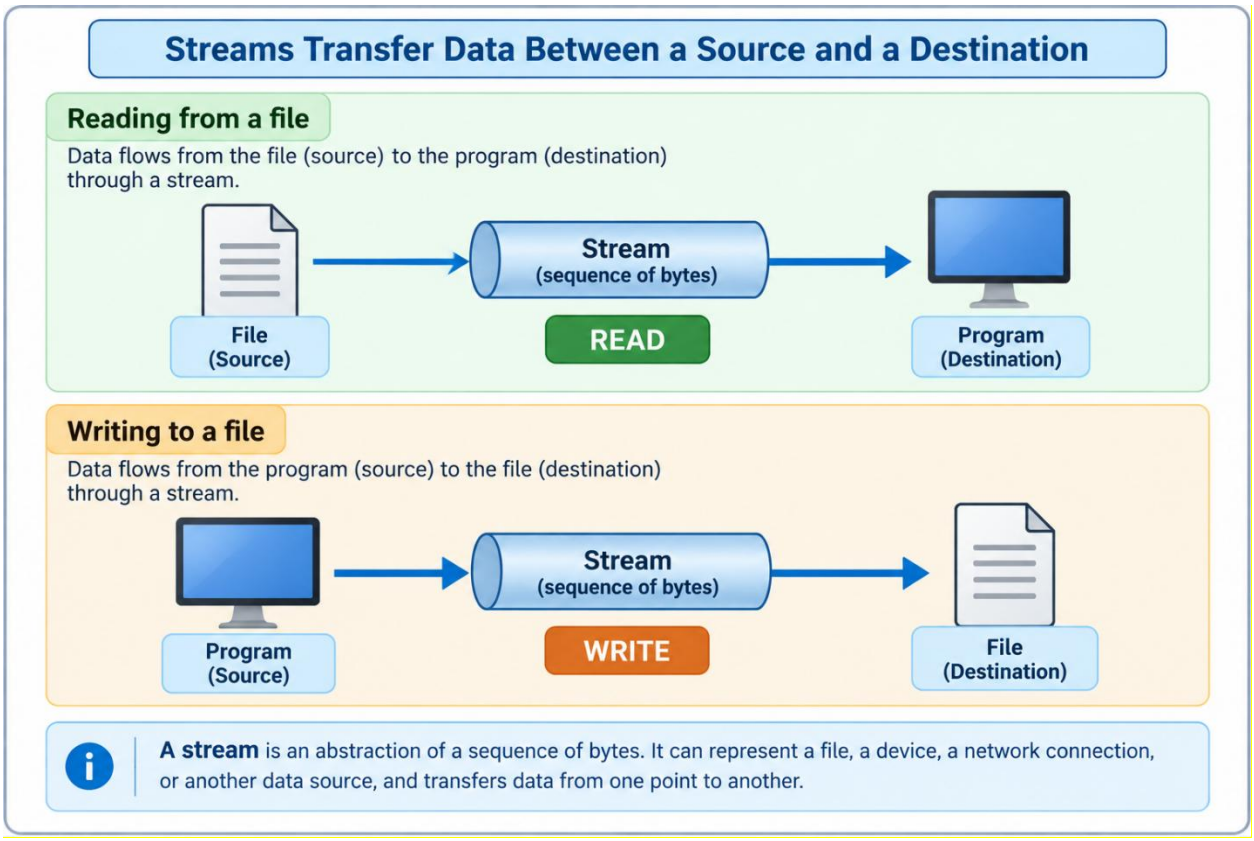
1. Understanding Streams

A **stream** is an abstraction representing a sequence of bytes. The underlying source or destination might be:

- a file;
- an input/output device;
- an interprocess communication pipe;
- a TCP/IP socket.

Streams transfer data from one point to another and can also participate in transformations such as compression or encryption.

Conceptually:



---

2. High-Level vs. Stream-Based File I/O

C# offers different levels of file access.

The **`File`** class provides convenient higher-level operations, while **`StreamReader`** and **`StreamWriter`** provide stream-based text I/O.

For example, reading an entire small text file can be as simple as:

```
string text = File.ReadAllText(path);
```

When reading incrementally, a **`StreamReader`** can be used:

```
using var reader = new StreamReader(path);
string? line = reader.ReadLine();
```

A practical distinction is:

Approach	Typical Purpose

`File`	Convenient whole-file operations
`StreamReader`	Read text progressively
`StreamWriter`	Write text through a stream
`FileStream`	Work directly with bytes

---

3. Why File I/O Requires Error Handling

File operations depend on resources outside the program's direct control. A file might:

- not exist;
- be inaccessible;
- be locked;
- have insufficient permissions.

Common I/O-related exceptions include **`FileNotFoundException`** and **`UnauthorizedAccessException`**. Unlike Java's checked-exception model, C# does not require the programmer to explicitly catch these exceptions.

If an exception is not handled by the application, it eventually propagates to the runtime.

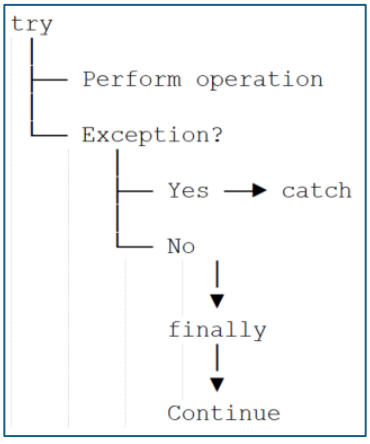
Warning

The fact that C# does not *require* you to catch an I/O exception does not mean exceptions should be ignored. File operations should be handled appropriately when failure is reasonably possible.

---

4. `try`, `catch`, and `finally`

C# uses try, catch, and finally to structure exception handling:



Their responsibilities are:

- **`try`** — contains operations that may fail;
- **`catch`** — handles exceptional circumstances;
- **`finally`** — performs cleanup whether an exception occurred or not.

A **`try/catch`** structure may be nested, while a particular **`try`** statement has at most one associated **`finally`** block.

Example

```
try
{
    string text = File.ReadAllText("data.txt");
    Console.WriteLine(text);
}
catch (FileNotFoundException ex)
{
    Console.WriteLine($"File not found: {ex.Message}");
}
catch (UnauthorizedAccessException ex)
{
    Console.WriteLine($"Access denied: {ex.Message}");
}
finally
{
    Console.WriteLine("File operation completed.");
}
```

The **`finally`** block executes whether reading succeeds or an exception is caught.

Tip

When the program can respond meaningfully to a specific exception, catching the specific exception type communicates intent more clearly than catching every exception as **`Exception`**.

---

5. I/O Resource Management

Streams use resources that must be released when they are no longer needed. The material identifies two principal approaches:

1. manually call **`Dispose()`**, commonly from **`finally`**;
2. use **`using`** so disposal occurs automatically.

Methods of the **`File`** class that perform complete operations internally manage their underlying resources automatically.

Why does this matter?

If a file resource remains open, the file may remain **locked**, preventing another operation or application from using it.

---

6. `IDisposable` and `Dispose()`

Streams implement **`IDisposable`**. This means they expose a **`Dispose()`** operation for releasing resources when the stream is finished.

A manual approach could look like:

```
StreamReader? reader = null;

try
{
    reader = new StreamReader("data.txt");
    Console.WriteLine(reader.ReadToEnd());
}
finally
{
    reader?.Dispose();
}
```

uses the null-conditional operator. **`Dispose()`** is called only if **`reader`** is not **`null`**.

Although this is valid, C# provides a more convenient mechanism: **`using`**.

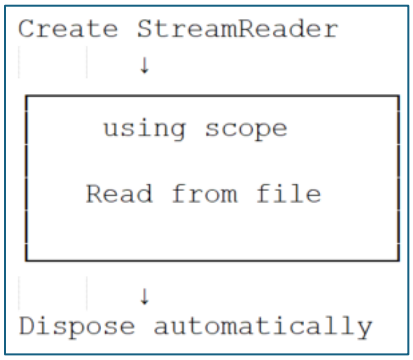
---

7. The `using` Statement

A **`using`** statement defines a scope in which an **`IDisposable`** object is used. When execution leaves that scope, the object is disposed.

```
using (var reader = new StreamReader("data.txt"))
{
    Console.WriteLine(reader.ReadToEnd());
}
```

Conceptually:



This avoids having to explicitly call:

```
reader.Dispose();
```

Tip

Prefer automatic resource management when possible. It reduces the risk of accidentally leaving files open.

---

8. The `using` Declaration

C# 8 introduced a shorter **using declaration**:

```
using var reader = new StreamReader(path);

Console.WriteLine(reader.ReadToEnd());
```

The object is automatically disposed at the end of the enclosing scope.

Compare the two forms:

```
// using statement
using (var reader = new StreamReader(path))
{
    Console.WriteLine(reader.ReadToEnd());
}
```

And:

```
// using declaration
using var reader = new StreamReader(path);

Console.WriteLine(reader.ReadToEnd());
```

Both provide automatic resource management; their scopes are expressed differently.

---

9. The `File` Class

The **`File`** class provides **static methods** for common operations on individual files, including creating, copying, deleting, moving, opening, reading, and writing files. It can also assist in creating **`FileStream`** objects.

Because the methods are static, you typically write:

```
File.Exists(path);
File.ReadAllText(path);
File.Copy(source, destination);
```

rather than constructing a **`File`** object.

---

10. Reading an Entire File with `ReadAllText()`

`File.ReadAllText()`:

1. opens the file;
2. reads its entire text;
3. closes the file;
4. returns the content as one **`string`**.

Step-by-Step Practice

Create a text file named:

`message.txt`

Add:

```
Welcome to C# File I/O.
This is our first file.
```

Then:

```
using System.Text;

string path = "message.txt";

string text = File.ReadAllText(path, Encoding.UTF8);

Console.WriteLine(text);
```

The important line is:

```
string text = File.ReadAllText(path, Encoding.UTF8);
```

The entire file becomes a single **`string`**.

---

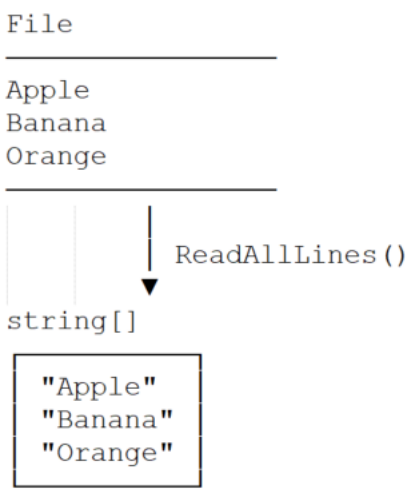
11. Reading Lines with `ReadAllLines()`

`File.ReadAllLines()` reads the complete file but returns its lines as a **`string[]`**.

```
string[] lines = File.ReadAllLines(path);

foreach (string line in lines)
{
    Console.WriteLine(line);
}
```

Conceptually:



This is convenient when each line represents a separate record or piece of information.

Warning — Large Files

`ReadAllLines()` loads all lines into memory at once. The material specifically cautions against using it for very large files.

---

12. Creating and Writing a Text File

`File.CreateText()` creates or opens a UTF-8 text file and returns a **`StreamWriter`**.

If the file already exists, its existing contents are overwritten.

```
using (StreamWriter writer = File.CreateText("cars.txt"))
{
    writer.WriteLine("Volvo");
    writer.WriteLine("Toyota");
    writer.WriteLine("Honda");
}
```

After leaving the **`using`** block, the writer is disposed and the file is closed.

Warning — Overwriting

`CreateText()` overwrites the existing contents of the file. Make sure this behavior is intended before using it with an existing file.

---

13. Checking Whether a File Exists

Before attempting certain operations, you can test whether a file exists:

```
if (File.Exists(path))
{
    Console.WriteLine("The file exists.");
}
```

You can also retrieve file timestamps:

```
if (File.Exists(path))
{
    Console.WriteLine(File.GetCreationTime(path));
    Console.WriteLine(File.GetLastWriteTime(path));
    Console.WriteLine(File.GetLastAccessTime(path));
}
```

These methods expose the creation, last-write, and last-access times associated with the file.

---

14. Copying Files

`File.Copy()` copies an existing file to another location.

```
File.Copy("fileA.txt", "fileB.txt");
```

By default, this does not overwrite an existing destination file.

To permit overwriting:

```
File.Copy("fileA.txt", "fileB.txt", true);
```

The third argument controls overwrite behavior:

- **`false`** → **do not overwrite**
- **`true`** → **overwrite existing destination**

15. Other `File` Operations

The **`File`** class contains many additional operations for working with files, including creating, copying, appending, moving, reading/writing text, and working with bytes.

The key distinction for this section is:

```
Simple complete file operation
            ↓
          File


More direct stream-based I/O
            ↓
StreamReader / StreamWriter / FileStream
```

---

16. Writing Text with `StreamWriter`

`StreamWriter` writes characters to a stream using a character encoding. Its default encoding is UTF-8.

```
string path = "notes.txt";

using (StreamWriter writer = new StreamWriter(path))
{
    writer.WriteLine("Today is a beautiful day.");
}
```

The sequence is:

C# string
↓
StreamWriter
↓
Encoding
↓
File

Creating a **`StreamWriter`** this way writes to the specified file; the constructor form shown here overwrites the existing file.

---

17. Reading Text with `StreamReader`

`StreamReader` reads characters from a byte stream and uses UTF-8 by default.

A line-oriented example is:

```
using (var reader = new StreamReader(path))
{
    while (reader.Peek() >= 0)
    {
        Console.WriteLine(reader.ReadLine());
    }
}
```

`Peek()` examines the next available character **without consuming it**.

It returns:

`>= 0` → **more characters are available**

`-1` → **end of stream**

`ReadLine()` then consumes and returns the next line.

---

18. Practical Lab — Write and Read a Text File

This exercise combines resource management, **`StreamWriter`**, and **`StreamReader`**.

Step 1 — Create the Project

Create a Console App named:

`FileIOPractice`

Step 2 — Define a Path

```
string path = "students.txt";
```

Step 3 — Write Data

```
using (StreamWriter writer = new StreamWriter(path))
{
    writer.WriteLine("Alex,85");
```

```
    writer.WriteLine("Maria,92");
    writer.WriteLine("Sam,78");
}
```

Step 4 — Read the Data

```
using (StreamReader reader = new StreamReader(path))
{
    while (reader.Peek() >= 0)
    {
        Console.WriteLine(reader.ReadLine());
    }
}
```

Step 5 — Run the Application

The output should be:

```
Alex,85

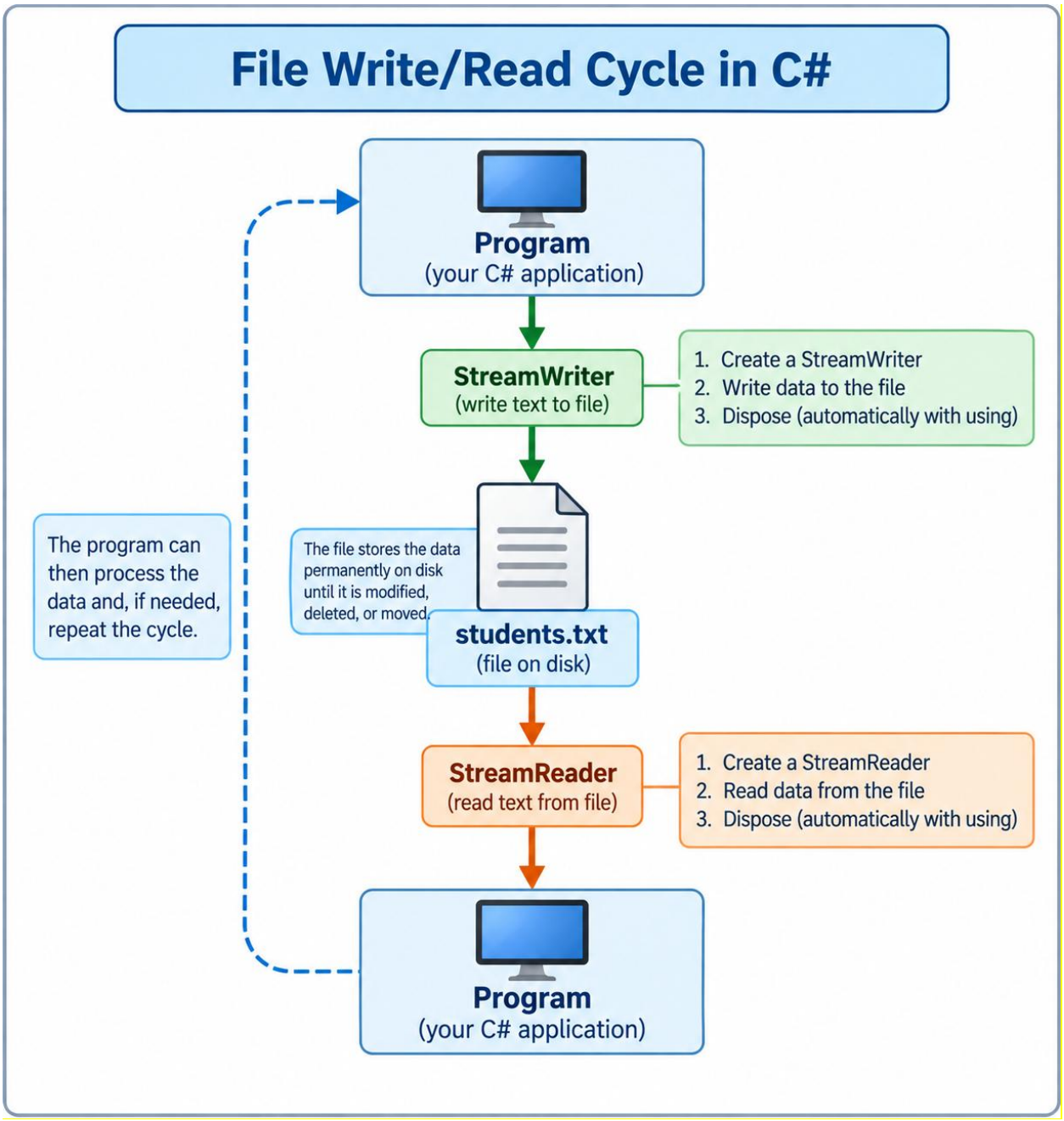
Maria,92

Sam,78
```

Step 6 — Locate the File

Find `students.txt` and inspect its contents.

This demonstrates the complete cycle:



---

19. `FileStream` — Working with Bytes

`StreamReader` and **`StreamWriter`** work with **text**. **`FileStream`** operates at the **byte level** and supports both reading and writing.

Example:

```
using System.Text;

string path = "authors.txt";

using (var stream = new FileStream(path, FileMode.Append))
{
    string text = "Фёдор Михайлович Достоевский\n";

    byte[] bytes = new UTF8Encoding().GetBytes(text);

    stream.Write(bytes, 0, bytes.Length);
}
```

Three operations are important here.

First: **`FileMode.Append`** opens the file so new data is added to the end.

Second: **`GetBytes(text)`** converts the characters into UTF-8 encoded bytes.

Finally: **`stream.Write(bytes, 0, bytes.Length);`** writes those bytes to the file.

Conceptually:

```
string
  ↓
UTF-8 Encoding
  ↓
byte[]
  ↓
FileStream
  ↓
File
```

Note

Encoding is the bridge between textual characters and the bytes stored in a file.

---

20. Working with Directories

File I/O also includes directory management.

The static **`Directory`** class provides methods for creating, moving, and enumerating directories and subdirectories.

Creating Directories

```
Directory.CreateDirectory("temp");
Directory.CreateDirectory("newdir");
```

Moving/Renaming a Directory

```
Directory.Move("temp", "temporary");
```

Here, the directory originally named **`temp`** becomes **`temporary`**.

---

21. DirectoryInfo

`DirectoryInfo` represents a particular directory and exposes information and operations associated with it.

Example:

```
string path = @"C:\Users\Student\Documents";

DirectoryInfo directory = new DirectoryInfo(path);

string[] files = Directory.GetFiles(path);
DirectoryInfo[] directories = directory.GetDirectories();
```

Display the subdirectories:

```
foreach (DirectoryInfo subDirectory in directories)
{
    Console.WriteLine(subDirectory.Name);
}
```

Display the files:

```
foreach (string fileName in files)
{
    Console.WriteLine(fileName);
}
```

This demonstrates the difference between:

```
Directory
    ↓
Static operations

DirectoryInfo
    ↓
Object representing a specific directory
```

---

22. Practical Lab — Explore a Directory

Step 1 — Choose a Directory

For example:

```
string path = @"C:\Temp";
```

Use a directory that actually exists on your computer.

Step 2 — Create a `DirectoryInfo`

```
DirectoryInfo directory = new DirectoryInfo(path);
```

Step 3 — Retrieve Subdirectories

```
DirectoryInfo[] directories =
    directory.GetDirectories();
```

Step 4 — Retrieve Files

```
string[] files = Directory.GetFiles(path);
```

Step 5 — Display Subdirectories

```
Console.WriteLine("DIRECTORIES");

foreach (DirectoryInfo subDirectory in directories)
{
    Console.WriteLine(subDirectory.Name);
}
```

Step 6 — Display Files

```
Console.WriteLine("\nFILES");

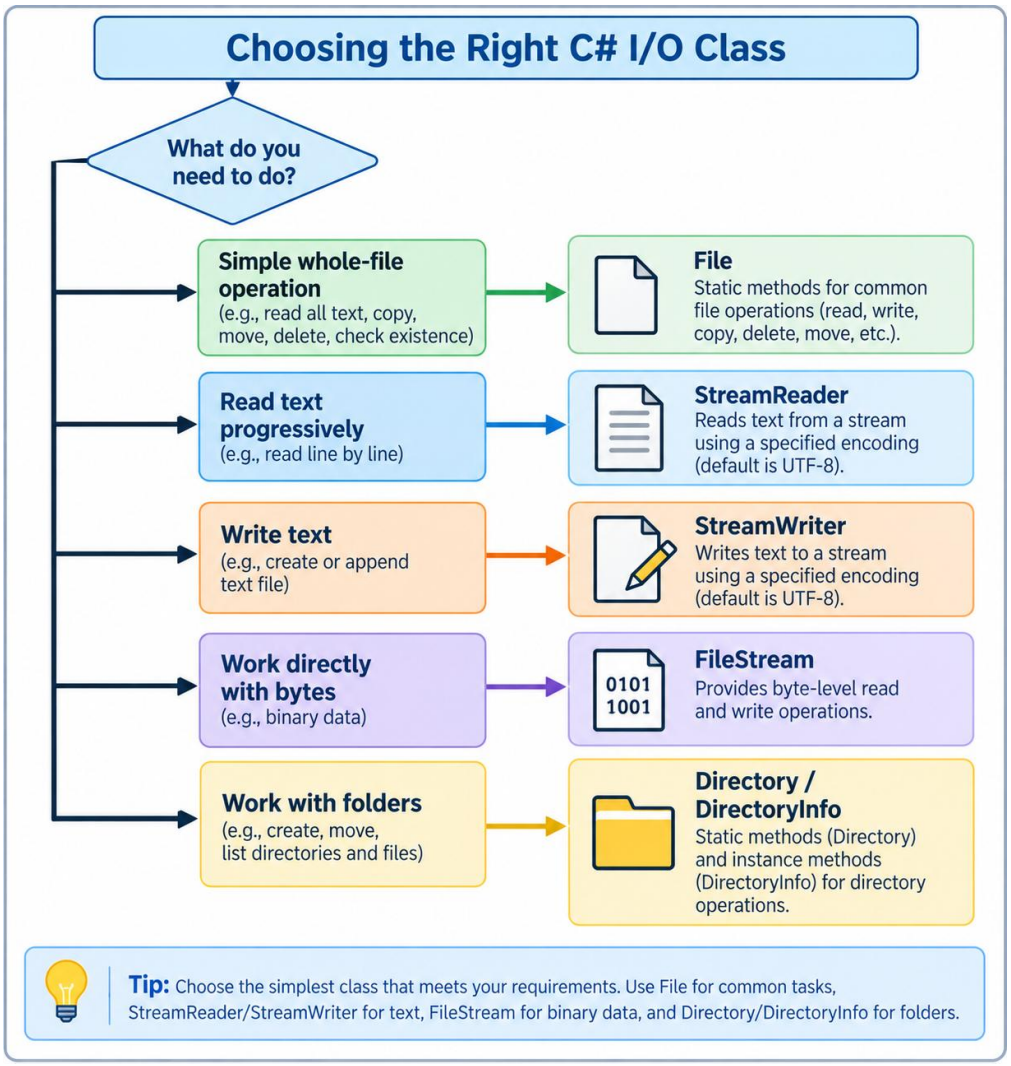
foreach (string file in files)
{
    Console.WriteLine(file);
}
```

Run the application and compare its output with the actual contents of the directory.

---

23. Choosing the Appropriate I/O Approach

The classes in this section overlap, but they serve different purposes:



---

24. Complete Practice — Simple File Manager

This final exercise combines the major concepts without introducing additional topics.

Step 1 — Create a Directory

string directory = "data";

```
Directory.CreateDirectory(directory);
```

Step 2 — Define a File

```
string path = Path.Combine(directory, "notes.txt");
```

Step 3 — Write Content

```
using (StreamWriter writer = new StreamWriter(path))
{
    writer.WriteLine("C# File I/O");
    writer.WriteLine("Working with files and streams.");
}
```

Step 4 — Verify the File

```
if (File.Exists(path))
{
    Console.WriteLine("File created successfully.");
}
```

Step 5 — Display File Information

```
Console.WriteLine($"Created: {File.GetCreationTime(path)}");
Console.WriteLine($"Modified: {File.GetLastWriteTime(path)}");
```

Step 6 — Read the File

```
using (StreamReader reader = new StreamReader(path))
{
    while (reader.Peek() >= 0)
    {
        Console.WriteLine(reader.ReadLine());
    }
}
```

Step 7 — Copy the File

```
string copyPath =
    Path.Combine(directory, "notes-backup.txt");

File.Copy(path, copyPath, true);
```

Step 8 — Verify the Result

Check that the directory now contains:

```
data
├── notes.txt
└── notes-backup.txt
```

This exercise demonstrates the core workflow:

```
Create Directory
        ↓
Create / Write File
        ↓
Check File
        ↓
Inspect Properties
        ↓
Read File
        ↓
Copy File
        ↓
Verify Result
```

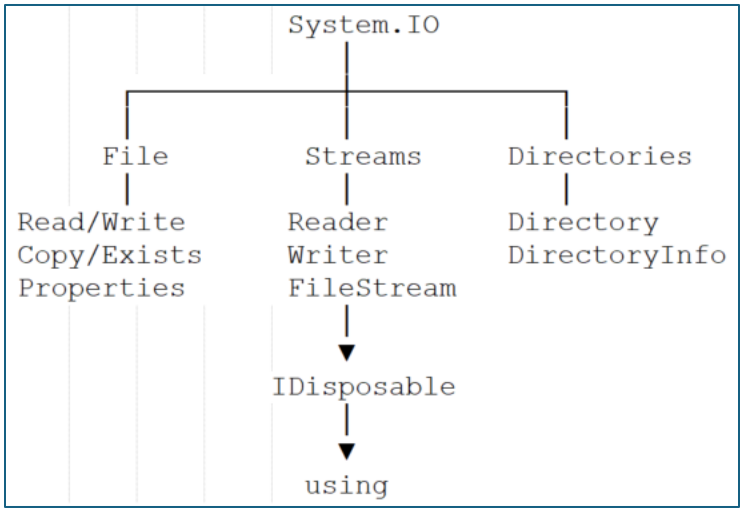
---

25. Key Concepts to Remember

The central abstraction behind C# file I/O is the **stream**—data moves between a source and destination as a sequence of bytes. Text-oriented classes such as **`StreamReader`** and **`StreamWriter`** provide a convenient layer over those byte streams, while **`FileStream`** provides direct byte-level access.

For straightforward whole-file operations, the static **`File`** class is usually simpler. When explicitly working with streams, resource lifetime matters: streams implement **`IDisposable`**, and **`using`** provides a convenient way to ensure that the resource is released when it is no longer required.

The practical relationship among the major concepts is:



The main skill is choosing the appropriate abstraction: **`File` for convenient complete operations, `StreamReader/StreamWriter` for text streams, `FileStream` for bytes, and `Directory/DirectoryInfo` for folders**, while managing resources and I/O failures correctly.